

Standalone Boot — Official Xilinx SREC Bootloader

This guide walks through AMD/Xilinx's **official** standalone-boot method for the MicroBlaze-V MCU: the `srec_spi_boot loader` application template shipped with Vitis. It is the vendor-supported alternative to this repo's custom UART bootloader ([Standalone-Boot-Mode](#)).

Everything below was **verified on the board** (Vitis 2025.2, MicroBlaze-V, Cmod A7-35T): the official bootloader read an SREC image from QSPI flash, copied the application into SRAM, and jumped to it — the app's banner printed on the USB serial console. It was done non-destructively, alongside the existing custom bootloader.

When to use which

	Official <code>srec_spi_boot loader</code>	This repo's custom UART bootloader
Vendor supported	Yes (Vitis template)	No (hand-written)
App format in flash	SREC (Motorola S-record text)	Raw binary + 16-byte header
Update the app over UART, no JTAG	No	Yes (<code>tools/upload.py</code>)
Update the app	re-write flash over JTAG (Vitis Program Flash)	serial upload, or JTAG
Integrity check	none by default	CRC32 (per-chunk + whole-image + read-back)
Force-stay-in-bootloader button	no	yes (BTN1)
Runs from	BRAM (embedded in bitstream)	BRAM (embedded in bitstream)
Copies app to	wherever the SREC addresses say (→ SRAM)	SRAM <code>0x6000_0000</code>

Rule of thumb: use the official one if you want a vendor-supported flow and JTAG-only updates are fine. Keep the custom one if you want the "plug-in-USB, update firmware with no JTAG" experience — the official bootloader cannot do that.

How it works

Identical architecture to the custom bootloader — only the app format and the update path differ:

```
power-on → FPGA configures from flash → BRAM comes up holding the bootloader
          → bootloader initializes the QSPI flash controller
          → reads the SREC text from flash @ FLASH_IMAGE_BASEADDR
```

- decodes each S-record, copies its bytes to the address it names
- jumps to the SREC's start address (S7/S8/S9 record)

Because the copy destinations come from the SREC itself, **the application's linker script decides where it runs** — link it to SRAM (`0x6000_0000`) and the SREC records place it there. The bootloader just honors the addresses.

Board-specific facts (verified — read before you start)

1. **The flash controller is auto-selected correctly under SDT.** Vitis 2025.2 uses the System Device Tree flow, where the template's `bootloader.c` picks `XPAR_AXI_QUAD_SPI_0_BASEADDR` — which is this board's flash controller. You do **not** have to fix the "which SPI" problem by hand. (Under the legacy flow it would use `XPAR_SPI_0_DEVICE_ID`, which on this board names the *external* SPI header — wrong. SDT saves you here.)
2. **FLASH_IMAGE_BASEADDR is the one thing you must set** (`src/blconfig.h`). It is the flash offset where the SREC image lives. Pick a region that does not collide with the bitstream (`0x00_0000–0x21_FFFF`) or, if the custom bootloader is also present, its app slot (`0x30_0000`). This guide uses `0x00340000` .
3. **Keep VERBOSE OFF.** The template's progress/debug prints go through `init_stdout()`, which on this board does **not** program the 16550 UART divisor — so `XUartNs550_SendByte` spins forever waiting for a TX-ready that never comes, and the bootloader hangs before it can jump. With `VERBOSE` undefined (the default) the bootloader is silent and boots fine. If you need diagnostics, add an explicit `XUartNs550_SetBaud(XPAR_UART_USB_BASEADDR, 100000000, 115200)` at the top of `main()` first. (Board-verified: with VERBOSE on, the CPU parks in `XUartNs550_SendByte`; with it off, the CPU reaches the application in SRAM.)
4. **Your application must set its own UART baud.** For the same reason, the loaded app should call `XUartNs550_SetBaud(..., 115200)` before printing (the course `app_template` `mcu_init()` already does this).

Step by step (Vitis GUI)

1. Create the bootloader component

Vitis → **File** → **New Component** → **Application** → pick your platform (built from `release/top_wrapper.xsa`, `standalone_microblaze_riscv_0`) → on the template page choose **"SREC SPI Bootloader"**. Vitis generates the sources.

2. Point it at the SREC location

Open `src/blconfig.h`, set:

```
#define FLASH_IMAGE_BASEADDR 0x00340000
```

(delete or silence the `#warning` line). Leave `VERBOSE` undefined.

3. Build → `srec_boot.elf`

Click Build. The bootloader links to BRAM (`0x0` , entry `0x0`), so it can copy the app into SRAM without overwriting itself.

4. Embed the bootloader into the bitstream (this is what makes it boot at power-on)

The bootloader must live in BRAM at configuration time. Merge its ELF into the bitstream's BRAM init and regenerate the flash image — same tool this repo uses for the custom one:

```
updatemem -force -meminfo <top_wrapper.mmi> -data srec_boot.elf \  
-bit release/top.bit -proc top_i/microblaze_riscv_0 -out boot_srec.bit  
write_cfgmem -format mcs -size 4 -interface SPIx4 -loadbit "up 0x0 boot_srec.bit" -  
file boot_srec.mcs
```

This step **replaces** the custom bootloader in the bitstream — it is the one destructive action. Skip it while you are only experimenting: you can prove the bootloader logic works by loading `srec_boot.elf` over JTAG instead (see "Verify over JTAG" below), which leaves the installed custom bootloader intact.

5. Build your application, linked to SRAM

Create your app component and use the SRAM linker script (`workspace-example/app_template/src/lscript.ld`) so it runs from `0x6000_0000` . Build → `app.elf` .

6. Convert the app to SREC

```
<riscv-objcopy> -O srec app.elf app.srec  
# objcopy lives under /opt/Xilinx/2025.2/gnu/riscv/.../bin/riscv32-amd-linux-gnu-  
objcopy
```

Check the record addresses are in SRAM: the `S3` lines start at `60000000...` and the terminating `S7` record's start address is `60000000` .

7. Write the SREC into flash

The Vitis **Program Flash** utility (or Vivado Hardware Manager → *Add Configuration Memory Device* → `mx25l3273f-spi-x1_x2_x4`) writes the image. To place raw SREC bytes at a specific offset without disturbing the rest of flash, build a partial MCS and program only that range:

```
# put the SREC bytes at FLASH_IMAGE_BASEADDR  
write_cfgmem -force -format mcs -size 4 -interface SPIx4 \  
-loaddata "up 0x00340000 app.srec" -file app_at_340000.mcs
```

```
# program ONLY the file's ranges (leaves the bitstream + any other app slot intact)
# create_hw_cfgmem ... mx25l3273f-spi-x1_x2_x4
# set PROGRAM.ADDRESS_RANGE {use_file}
# set PROGRAM.FILES {app_at_340000.mcs}
# program_hw_cfgmem
```

`ADDRESS_RANGE {use_file}` scopes the erase+program to the offsets present in the MCS — this is how the write stays non-destructive.

8. Power on

Power-cycle (or JPROGRAM the `boot_srec.bit` from step 4). The bootloader reads the SREC from `0x340000`, copies your app to SRAM, and jumps. Your app's banner appears on the USB serial console.

Verify over JTAG (non-destructive, no bitstream change)

To prove the bootloader logic without replacing the installed custom bootloader:

```
# 1. write the SREC to a free flash region (step 7 above)
# 2. load the FPGA design, then run the bootloader from RAM over JTAG:
python3 tools/jtag_run.py srec_boot.elf --bit release/top.bit
```

The bootloader runs from BRAM (loaded by JTAG), reads the SREC from flash, copies the app into SRAM, and jumps. Confirm with xsdb that the PC lands in SRAM (`0x6000_xxxx`) and the app's banner prints. This is exactly how this guide was validated on the board. A power-cycle afterward restores the normal (custom-bootloader) boot, since nothing in the bitstream or the custom app slot was touched.

Summary

The official `srec_spi_bootloader` is a real, supported standalone-boot path for this MicroBlaze-V MCU, and it works on this board with a single config change (`FLASH_IMAGE_BASEADDR`) plus one caveat (keep `VERBOSE` off). It gives you the "app in flash, boots at power-on" behavior. What it does **not** give you is the custom bootloader's serial firmware-update — so this repo keeps both: the official flow for a vendor-supported baseline, and the custom UART bootloader for the commercial-MCU update experience.